

Implementation of Scheduler History System Call in Linux

Project Report

Operating Systems [CS F372]

Semester II, 2025 – 2026



BITS Pilani
K K Birla Goa Campus

Course Instructor: Dr Biju K Raveendran

Submitted By

Group 36 – KernelX

Jaikumar Wath 2023A3PS0197G
Ayush Pareek 2023AAPS0609G

INDEX

1. Kernel Recompilation	3
1.1 Downloading Kernel Source	3
1.2 Extract the Kernel Source	4
1.3 Install Required Dependencies	5
1.4 Kernel Configuration	6
1.5 Kernel Compilation	6
1.6 Kernel Installation & verification	7
1.7 Error Handling	8
2. System Call Implementation	9
2.1 System Call Setup and Directory Creation	9
2.2 Implementation of System Call Logic	9
Data Structure and Buffer	10
Logging Scheduling Events	10
System Call Implementation	10
2.3 Modifications in Kernel Source Files	11
1. Makefile Modification	11
2. System Call Table Update	11
3. System Call Declaration	12
4. Scheduler Modification (core.c)	13
5. System Call Makefile	13
3. User space implementation	16
3.1 System Call Invocation	16
3.2 Compiling and Executing the User program	17

1. Kernel Recompilation

1.1 Downloading Kernel Source

```
root@Curse: /usr/src
(base) jai_wath_17@Curse:/usr/src$ sudo su
[sudo] password for jai_wath_17:
root@Curse:/usr/src# cd /usr/src
root@Curse:/usr/src# wget https://cdn.kernel.org/pub/linux/kernel/v5.x/linux-5.19.8.tar.xz
--2026-03-29 16:01:27-- https://cdn.kernel.org/pub/linux/kernel/v5.x/linux-5.19.8.tar.xz
Resolving cdn.kernel.org (cdn.kernel.org)... 2a04:4e42:59::432, 151.101.157.176
Connecting to cdn.kernel.org (cdn.kernel.org)[2a04:4e42:59::432]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 131632044 (126M) [application/x-xz]
Saving to: 'linux-5.19.8.tar.xz'

linux-5.19.8.tar.xz      100%[=====] 125.53M  2.65MB/s   in 49s

2026-03-29 16:02:17 (2.55 MB/s) - 'linux-5.19.8.tar.xz' saved [131632044/131632044]

root@Curse:/usr/src#
```

The Linux kernel source (version **5.19.8**) was downloaded using:

cd /usr/src

sudo su

wget <https://cdn.kernel.org/pub/linux/kernel/v5.x/linux-5.19.8.tar.xz>

The file **linux-5.19.8.tar.xz** was successfully downloaded into the **/usr/src** directory, and the working directory was set to **/usr/src**.

1.2 Extract the Kernel Source

```
root@Curse: /usr/src/linux-5.19.8
linux-5.19.8/tools/virtio/vringh_test.c
linux-5.19.8/tools/virtio/xen/
linux-5.19.8/tools/virtio/xen/xen.h
linux-5.19.8/tools/vm/
linux-5.19.8/tools/vm/.gitignore
linux-5.19.8/tools/vm/Makefile
linux-5.19.8/tools/vm/page-types.c
linux-5.19.8/tools/vm/page_owner_sort.c
linux-5.19.8/tools/vm/slabinfo-gnuplot.sh
linux-5.19.8/tools/vm/slabinfo.c
linux-5.19.8/tools/mmi/
linux-5.19.8/tools/mmi/Makefile
linux-5.19.8/tools/mmi/dell-smbios-example.c
linux-5.19.8/user/
linux-5.19.8/user/.gitignore
linux-5.19.8/user/Kconfig
linux-5.19.8/user/Makefile
linux-5.19.8/user/default_cpio_list
linux-5.19.8/user/dummy-include/
linux-5.19.8/user/dummy-include/stdbool.h
linux-5.19.8/user/dummy-include/stdlib.h
linux-5.19.8/user/gen_init_cpio.c
linux-5.19.8/user/gen_initramfs.sh
linux-5.19.8/user/include/
linux-5.19.8/user/include/.gitignore
linux-5.19.8/user/include/Makefile
linux-5.19.8/user/include/headers_check.pl
linux-5.19.8/user/initramfs_data.S
linux-5.19.8/virt/
linux-5.19.8/virt/Makefile
linux-5.19.8/virt/kvm/
linux-5.19.8/virt/kvm/Kconfig
linux-5.19.8/virt/kvm/Makefile.kvm
linux-5.19.8/virt/kvm/async_pf.c
linux-5.19.8/virt/kvm/async_pf.h
linux-5.19.8/virt/kvm/binary_stats.c
linux-5.19.8/virt/kvm/coalesced_mmio.c
linux-5.19.8/virt/kvm/coalesced_mmio.h
linux-5.19.8/virt/kvm/dirty_ring.c
linux-5.19.8/virt/kvm/eventfd.c
linux-5.19.8/virt/kvm/irqchip.c
linux-5.19.8/virt/kvm/kvm_main.c
linux-5.19.8/virt/kvm/kvm_mm.h
linux-5.19.8/virt/kvm/pfncache.c
linux-5.19.8/virt/kvm/vfio.c
linux-5.19.8/virt/kvm/vfio.h
linux-5.19.8/virt/lib/
linux-5.19.8/virt/lib/Kconfig
linux-5.19.8/virt/lib/Makefile
linux-5.19.8/virt/lib/irqbypass.c
root@Curse: /usr/src/linux-5.19.8#
```

The kernel source was extracted using:

```
tar -xvf linux-5.19.8.tar.xz  
cd linux-5.19.8
```

This created a directory named **linux-5.19.8** containing the kernel source code, and the working directory was changed to the extracted kernel source.

1.3 Install Required Dependencies

```
root@Curse:/usr/src/linux-5.19.8
linux-5.19.8/virt/lib/
linux-5.19.8/virt/lib/Kconfig
linux-5.19.8/virt/lib/Makefile
linux-5.19.8/virt/lib/irqbypass.c
root@Curse:/usr/src/linux-5.19.8# sudo apt update
sudo apt install flex bison libssl-dev libelf-dev libncurses-dev bc gawk dwarves
Hit:1 http://in.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:4 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:5 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,846 kB]
Get:6 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [178 kB]
Get:7 http://in.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:8 http://in.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,665 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:11 http://in.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,665 kB]
Get:12 http://in.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:15 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Get:17 http://in.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [13.2 kB]
Get:18 http://in.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:19 http://in.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7,352 B]
Get:20 http://in.archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [216 B]
Get:21 http://in.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [13.2 kB]
Get:22 http://in.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]
Get:23 http://in.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,665 kB]
Get:24 http://in.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Fetched 2,547 kB in 2min 22s (17.9 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
3 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
flex is already the newest version (2.6.4-8.2build1).
bison is already the newest version (2:3.8.2+dfsg-1build2).
libssl-dev is already the newest version (3.0.13-0ubuntu3.7).
libelf-dev is already the newest version (0.190-1.1ubuntu0.1).
libncurses-dev is already the newest version (6.4+20240113-1ubuntu2).
bc is already the newest version (1.07.1-3ubuntu4).
bc set to manually installed.
gawk is already the newest version (1:5.2.1-2build3).
dwarves is already the newest version (1.25-0ubuntu3).
0 upgraded, 0 newly installed, 0 to remove and 3 not upgraded.
root@Curse:/usr/src/linux-5.19.8#
```

Before compiling the Linux kernel, it is necessary to install the required development tools and libraries. These dependencies are essential for building the kernel successfully without errors.

The following commands were executed to update the package list and install the required packages:

sudo apt update

sudo apt install flex bison libssl-dev libelf-dev bc build-essential dwarves

1.4 Kernel Configuration

```
root@Curse:/usr/src/linux-5.19.8# make olddefconfig
#
# configuration written to .config
#
root@Curse:/usr/src/linux-5.19.8#
```

Before compiling the kernel, the configuration file must be prepared. This step ensures that the kernel is configured according to the system requirements.

The existing system configuration was reused using the following command:

make olddefconfig

This command updates the configuration file based on the current system defaults and ensures compatibility with the kernel version being compiled.

1.5 Kernel Compilation

```
LD [M] sound/soc/intel/boards/snd-soc-sst-haswell.ko
LD [M] sound/soc/intel/boards/snd-soc-sst-sof-pcm512x.ko
LD [M] sound/soc/intel/boards/snd-soc-sst-sof-wm8804.ko
LD [M] sound/soc/intel/catpt/snd-soc-catpt.ko
LD [M] sound/soc/intel/common/snd-soc-acpi-intel-match.ko
LD [M] sound/soc/snd-soc-acpi.ko
LD [M] sound/soc/snd-soc-core.ko
LD [M] sound/soc/sof/amd/snd-sof-amd-acp.ko
LD [M] sound/soc/sof/amd/snd-sof-amd-renoir.ko
LD [M] sound/soc/sof/intel/snd-sof-acpi-intel-bdw.ko
LD [M] sound/soc/sof/intel/snd-sof-acpi-intel-byt.ko
LD [M] sound/soc/sof/intel/snd-sof-intel-atom.ko
LD [M] sound/soc/sof/intel/snd-sof-intel-hda-common.ko
LD [M] sound/soc/sof/intel/snd-sof-intel-hda.ko
LD [M] sound/soc/sof/intel/snd-sof-pci-intel-apl.ko
LD [M] sound/soc/sof/intel/snd-sof-pci-intel-cn1.ko
LD [M] sound/soc/sof/intel/snd-sof-pci-intel-icl.ko
LD [M] sound/soc/sof/intel/snd-sof-pci-intel-tgl.ko
LD [M] sound/soc/sof/intel/snd-sof-pci-intel-tng.ko
LD [M] sound/soc/sof/snd-sof-acpi.ko
LD [M] sound/soc/sof/snd-sof-pci.ko
LD [M] sound/soc/sof/snd-sof-probes.ko
LD [M] sound/soc/sof/snd-sof-utils.ko
LD [M] sound/soc/sof/snd-sof.ko
LD [M] sound/soc/sof/xtensa/snd-sof-xtensa-dsp.ko
LD [M] sound/soc/xilinx/snd-soc-xlnx-formatter-pcm.ko
LD [M] sound/soc/xilinx/snd-soc-xlnx-i2s.ko
LD [M] sound/soc/xilinx/snd-soc-xlnx-spdif.ko
LD [M] sound/soc/xtensa/snd-soc-xtfpga-i2s.ko
LD [M] sound/soundcore.ko
LD [M] sound/synth/emux/snd-emux-synth.ko
LD [M] sound/synth/snd-util-mem.ko
LD [M] sound/usb/6fire/snd-usb-6fire.ko
LD [M] sound/usb/bcd2000/snd-bcd2000.ko
LD [M] sound/usb/caiaq/snd-usb-caiaq.ko
LD [M] sound/usb/hiface/snd-usb-hiface.ko
LD [M] sound/usb/line6/snd-usb-line6.ko
LD [M] sound/usb/line6/snd-usb-pod.ko
LD [M] sound/usb/line6/snd-usb-podhd.ko
LD [M] sound/usb/line6/snd-usb-toneport.ko
LD [M] sound/usb/line6/snd-usb-variax.ko
LD [M] sound/usb/misc/snd-ua101.ko
LD [M] sound/usb/snd-usb-audio.ko
LD [M] sound/usb/snd-usbmidi-lib.ko
LD [M] sound/usb/usx2y/snd-usb-usx2y1221.ko
LD [M] sound/usb/usx2y/snd-usb-usx2y.ko
LD [M] sound/virtio/virtio_snd.ko
LD [M] sound/x86/snd-hdmi-lpe-audio.ko
LD [M] sound/xen/snd_xen_front.ko
LD [M] virt/lib/lrqbypass.ko
root@Curse:/usr/src/linux-5.19.8#
```

After configuring the kernel, the next step was to recompile the Linux kernel. This process builds the kernel image and all necessary modules required for system operation.

The kernel was compiled using the following command:

make -j\$(nproc)

This command utilizes all available CPU cores, thereby speeding up the compilation process. During compilation, various kernel components such as drivers, modules, and core subsystems were built successfully.

The compilation process completed without issues after proper configuration.

1.6 Kernel Installation & verification

```
root@Curse: /usr/src/linux-5.19.8
SIGN /lib/modules/5.19.8/kernel/sound/usb/snd-usb-audio.ko
ZSTD /lib/modules/5.19.8/kernel/sound/usb/snd-usb-audio.ko.zst
INSTALL /lib/modules/5.19.8/kernel/sound/usb/snd-usbmidi-lib.ko
SIGN /lib/modules/5.19.8/kernel/sound/usb/snd-usbmidi-lib.ko
ZSTD /lib/modules/5.19.8/kernel/sound/usb/snd-usbmidi-lib.ko.zst
INSTALL /lib/modules/5.19.8/kernel/sound/usb/snd-usbmidi-lib.ko.zst
SIGN /lib/modules/5.19.8/kernel/sound/usb/usb2y/snd-usb-us1221.ko
SIGN /lib/modules/5.19.8/kernel/sound/usb/usb2y/snd-usb-us1221.ko
ZSTD /lib/modules/5.19.8/kernel/sound/usb/usb2y/snd-usb-us1221.ko.zst
INSTALL /lib/modules/5.19.8/kernel/sound/usb/usb2y/snd-usb-usx2y.ko
SIGN /lib/modules/5.19.8/kernel/sound/usb/usb2y/snd-usb-usx2y.ko
ZSTD /lib/modules/5.19.8/kernel/sound/usb/usb2y/snd-usb-usx2y.ko.zst
INSTALL /lib/modules/5.19.8/kernel/sound/virtio/virtio_snd.ko
SIGN /lib/modules/5.19.8/kernel/sound/virtio/virtio_snd.ko
ZSTD /lib/modules/5.19.8/kernel/sound/virtio/virtio_snd.ko.zst
INSTALL /lib/modules/5.19.8/kernel/sound/x86/snd-hdmi-lpe-audio.ko
SIGN /lib/modules/5.19.8/kernel/sound/x86/snd-hdmi-lpe-audio.ko
ZSTD /lib/modules/5.19.8/kernel/sound/x86/snd-hdmi-lpe-audio.ko.zst
INSTALL /lib/modules/5.19.8/kernel/sound/xen/snd_xen_front.ko
SIGN /lib/modules/5.19.8/kernel/sound/xen/snd_xen_front.ko
ZSTD /lib/modules/5.19.8/kernel/sound/xen/snd_xen_front.ko.zst
INSTALL /lib/modules/5.19.8/kernel/virt/lib/irqbypass.ko
SIGN /lib/modules/5.19.8/kernel/virt/lib/irqbypass.ko
ZSTD /lib/modules/5.19.8/kernel/virt/lib/irqbypass.ko.zst
DEPMOD /lib/modules/5.19.8
INSTALL /boot
run-parts: executing /etc/kernel/postinst.d/initramfs-tools 5.19.8 /boot/vmlinuz-5.19.8
update-initramfs: Generating /boot/initrd.img-5.19.8
run-parts: executing /etc/kernel/postinst.d/unattended-upgrades 5.19.8 /boot/vmlinuz-5.19.8
run-parts: executing /etc/kernel/postinst.d/update-notifier 5.19.8 /boot/vmlinuz-5.19.8
run-parts: executing /etc/kernel/postinst.d/xx-update-initrd-links 5.19.8 /boot/vmlinuz-5.19.8
I: /boot/initrd.img-old is now a symlink to initrd.img-6.19.10
I: /boot/initrd.img is now a symlink to initrd.img-5.19.8
run-parts: executing /etc/kernel/postinst.d/zz-shim 5.19.8 /boot/vmlinuz-5.19.8
run-parts: executing /etc/kernel/postinst.d/zz-update-grub 5.19.8 /boot/vmlinuz-5.19.8
Sourcing file '/etc/default/grub'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.19.10
Found initrd image: /boot/initrd.img-6.19.10
Found linux image: /boot/vmlinuz-6.17.0-19-generic
Found initrd image: /boot/initrd.img-6.17.0-19-generic
Found linux image: /boot/vmlinuz-6.17.0-14-generic
Found initrd image: /boot/initrd.img-6.17.0-14-generic
Found linux image: /boot/vmlinuz-5.19.8
Found initrd image: /boot/initrd.img-5.19.8
Found memtest86+ 64bit EFI image: /boot/memtest86+64.efi
Warning: os-prober will be executed to detect other bootable partitions.
Its output will be used to detect bootable binaries on them and create new boot entries.
Found Windows Boot Manager on /dev/nvme0n1p10/EFI/Microsoft/Boot/bootmgfw.efi
Adding boot menu entry for UEFI Firmware Settings ...
done
root@Curse:/usr/src/linux-5.19.8#
```

After compiling the kernel, it was installed using:

make modules_install

make install

These commands install the modules in /lib/modules and the kernel in /boot.

The system also updates initramfs and GRUB to enable booting into the new kernel.

```
(base) jai_wath_17@Curse:~$ uname -r
5.19.8
(base) jai_wath_17@Curse:~$
```

After installation, the system was rebooted to load the newly installed kernel.

The kernel version was verified using the following command:

uname -r

The output confirmed that the system is running the newly compiled kernel version.

1.7 Error Handling

```
root@Curse: /usr/src/linux-5.19.8
CC [M] drivers/hid/hid-sensor-custom.o
CC [M] drivers/comedi/drivers/ni_routing/ni_device_routes/pci-6254.o
CC [M] drivers/comedi/drivers/ni_routing/ni_device_routes/pci-6259.o
CC [M] drivers/comedi/drivers/ni_routing/ni_device_routes/pci-6534.o
CC [M] drivers/comedi/drivers/ni_routing/ni_device_routes/pxie-6535.o
CC [M] drivers/comedi/drivers/ni_routing/ni_device_routes/pci-6602.o
CC [M] drivers/comedi/drivers/ni_routing/ni_device_routes/pci-6713.o
CC [M] drivers/comedi/drivers/ni_routing/ni_device_routes/pci-6723.o
CC [M] drivers/comedi/drivers/ni_routing/ni_device_routes/pci-6733.o
CC [M] drivers/comedi/drivers/ni_routing/ni_device_routes/pxi-6733.o
CC [M] drivers/comedi/drivers/ni_routing/ni_device_routes/pxie-6738.o
CC [M] drivers/comedi/drivers/ni_labpc_common.o
CC [M] drivers/comedi/drivers/ni_labpc_isadma.o
CC [M] drivers/comedi/drivers/comedi_8255.o
CC [M] drivers/comedi/drivers/8255.o
CC [M] drivers/comedi/drivers/ampc_dio200_common.o
CC [M] drivers/comedi/drivers/ampc_pc236_common.o
CC [M] drivers/comedi/drivers/das08.o
LD [M] drivers/hid/hid.o
LD [M] drivers/hid/hid-logitech.o
LD [M] drivers/hid/hid-picolcd.o
LD [M] drivers/hid/hid-uclogic.o
LD [M] drivers/comedi/drivers/ni_routing.o
LD [M] drivers/hid/hid-wimote.o
LD [M] drivers/hid/wacom.o
GEN .version
CHK include/generated/compile.h
LD vmlinux.o
MODPOST vmlinux.symvers
MODINFO modules.builtin.modinfo
GEN modules.builtin
CC .vmlinux.export.o
LD .tmp_vmlinux.btf
BTF .btf.vmlinux.bin.o
LD .tmp_vmlinux.kallsyms1
KSYM .tmp_vmlinux.kallsyms1.S
AS .tmp_vmlinux.kallsyms1.S
LD .tmp_vmlinux.kallsyms2
KSYM .tmp_vmlinux.kallsyms2.S
AS .tmp_vmlinux.kallsyms2.S
LD vmlinux
BTFIDS vmlinux
FAILED: load BTF from vmlinux: Invalid argument
make: *** [Makefile:1168: vmlinux] Error 255
make: *** Deleting file 'vmlinux'
root@Curse:/usr/src/linux-5.19.8# scripts/config --disable DEBUG_INFO_BTF
root@Curse:/usr/src/linux-5.19.8# make olddefconfig
#
# configuration written to .config
#
root@Curse:/usr/src/linux-5.19.8#
```

During kernel compilation, an error related to BTF (BPF Type Format) was encountered:
"FAILED: load BTF from vmlinux: Invalid argument"

This issue was resolved by disabling the `DEBUG_INFO_BTF` option using the following commands:

```
scripts/config --disable DEBUG_INFO_BTF  
make olddefconfig
```

After applying this fix, the kernel compilation proceeded successfully without further errors.

2. System Call Implementation

Scheduler History System Call

The system call `get_sched_history` is implemented to track the scheduling behavior of processes in the Linux kernel. It records when a process is scheduled in and scheduled out of the CPU.

Each scheduling event stores the timestamp, CPU core, event type, and process ID. The events are maintained in a fixed-size buffer inside the kernel.

The system call takes a process ID and returns its recent scheduling history to user space.

2.1 System Call Setup and Directory Creation

A new directory `sched_syscall` was created inside the Linux kernel source to store files related to the custom system call.

```
cd /usr/src/linux-5.19.8  
mkdir sched_syscall
```

```
(base) jai_wath_17@Curse:/usr/src/linux-5.19.8$ sudo su  
[sudo] password for jai_wath_17:  
root@Curse:/usr/src/linux-5.19.8# mkdir sched_syscall  
root@Curse:/usr/src/linux-5.19.8# ls  
arch  COPYING  Documentation  include  ipc  kernel  MAINTAINERS  modules.builtin  modules.order  README  scripts  System.map  virt  vmlinux.map  
block  CREDITS  drivers  init  Kbuild  lib  Makefile  modules.builtin.modinfo  Module.symvers  samples  security  tools  vmlinux  vmlinux.map  
certs  crypto  fs  io_uring  Kconfig  LICENSES  mm  modules-only.symvers  net  sched_syscall  sound  usr  vmlinux-gdb.py  vmlinux.o  
root@Curse:/usr/src/linux-5.19.8#
```

2.2 Implementation of System Call Logic

Inside the `sched_syscall` directory, the source file `sched_syscall.c` was created to implement the custom system call.

```
cd sched_syscall  
touch sched_syscall.c
```

This file contains the data structure, buffer management, and logic required to record scheduling events.

```
GNU nano 7.2 root@Curse: /home/jpl_wath/17/Documents/os_proj/user_space /usr/src/linux-5.19.8/sched_syscall/sched_syscall.c *
#include <linux/kernel.h>
#include <linux/syscalls.h>
#include <linux/sched.h>
#include <linux/timekeeping.h>
#include <linux/uaccess.h>
#include <linux/smp.h>

#define MAX_EVENTS 1000

struct sched_event {
    u64 timestamp;
    int cpu;
    int event; // 0 = IN, 1 = OUT
    pid_t pid;
};

static struct sched_event buffer[MAX_EVENTS];
static int index = 0;

void log_sched_event(pid_t pid, int event)
{
    buffer[index].timestamp = ktime_get_ns();
    buffer[index].cpu = smp_processor_id();
    buffer[index].event = event;
    buffer[index].pid = pid;

    index = (index + 1) % MAX_EVENTS;
}

SYSCALL_DEFINE3(get_sched_history,
                pid_t, pid,
                struct sched_event __user *, user_buf,
                int, max_events)
{
    int i, count = 0;

    for (i = 0; i < MAX_EVENTS && count < max_events; i++) {
        if (buffer[i].pid == pid) {
            if (copy_to_user(&user_buf[count],
                            &buffer[i],
                            sizeof(struct sched_event)))
                return -EFAULT;
            count++;
        }
    }

    return count;
}
```

Data Structure and Buffer

A structure named `sched_event` was defined to represent each scheduling event. It stores the timestamp, CPU core, event type, and process ID.

A fixed-size buffer was also created to store scheduling events inside the kernel. The buffer operates in a circular manner, ensuring that older events are overwritten when full.

Logging Scheduling Events

A function named `log_sched_event` was implemented to record scheduling events.

This function captures the current timestamp, CPU core, event type, and process ID, and stores them in the buffer. The buffer index is updated in a circular manner to continuously store recent events.

System Call Implementation

The system call `get_sched_history` was implemented using the `SYSCALL_DEFINE3` macro.

It takes a process ID, a user-space buffer, and a maximum event limit as input. The system call filters events corresponding to the given process and copies them safely to user space using `copy_to_user`. It returns the number of events retrieved.

2.3 Modifications in Kernel Source Files

To integrate the custom system call into the Linux kernel, several kernel source files were modified.

1. Makefile Modification

```
GNU nano 7.2 /usr/s
#
# INSTALL_DTBS_PATH specifies a prefix for relocations required by build roots.
# Like INSTALL_MOD_PATH, it isn't defined in the Makefile, but can be passed as
# an argument if needed. Otherwise it defaults to the kernel install path
#
export INSTALL_DTBS_PATH ?= $(INSTALL_PATH)/dtbs/$(KERNELRELEASE)
#
# INSTALL_MOD_PATH specifies a prefix to MODLIB for module directory
# relocations required by build roots. This is not defined in the
# makefile but the argument can be passed to make if needed.
#
MODLIB = $(INSTALL_MOD_PATH)/lib/modules/$(KERNELRELEASE)
export MODLIB

PHONY += prepare0

export extmod_prefix = $(if $(KBUILD_EXTMOD),$(KBUILD_EXTMOD)/)
export MODORDER := $(extmod_prefix)modules.order
export MODULES_NSDEPS := $(extmod_prefix)modules.nsdeps

ifeq ($(KBUILD_EXTMOD),)
core-y      += kernel/ certs/ mm/ fs/ ipc/ security/ crypto/ sched_syscall/
core-$(CONFIG_BLOCK) += block/
core-$(CONFIG_IO_URING) += io_uring/
```

The kernel `Makefile` was updated by adding the `sched_syscall/` directory to the `core-y` list:

`core-y += kernel/ certs/ mm/ fs/ ipc/ security/ crypto/ sched_syscall/`

This ensures that the system call implementation is compiled as part of the kernel build.

2. System Call Table Update

```
449      common  futex_waitv      sys_futex_waitv
450      common  set_mempolicy_home_node sys_set_mempolicy_home_node
451      common  get_sched_history  sys_get_sched_history
#
```

The file `syscall_64.tbl` was modified to register the new system call.

A new entry was added:

451 common get_sched_history sys_get_sched_history

This assigns a unique system call number and links it to its implementation.

3. System Call Declaration

```
GNU nano 7.2
asmlinkage long sys_ftruncate64(unsigned int fd, loff_t length);
#endif
asmlinkage long sys_fallocate(int fd, int mode, loff_t offset, loff_t len);
asmlinkage long sys_faccessat(int dfd, const char __user *filename, int mode);
asmlinkage long sys_faccessat2(int dfd, const char __user *filename, int mode,
                                int flags);
asmlinkage long sys_chdir(const char __user *filename);
asmlinkage long sys_fchdir(unsigned int fd);
asmlinkage long sys_chroot(const char __user *filename);
asmlinkage long sys_fchmod(unsigned int fd, umode_t mode);
asmlinkage long sys_fchmodat(int dfd, const char __user * filename,
                                umode_t mode);
asmlinkage long sys_fchownat(int dfd, const char __user *filename, uid_t user,
                                gid_t group, int flag);
asmlinkage long sys_fchown(unsigned int fd, uid_t user, gid_t group);
asmlinkage long sys_openat(int dfd, const char __user *filename, int flags,
                                umode_t mode);
asmlinkage long sys_openat2(int dfd, const char __user *filename,
                                struct open_how *how, size_t size);
asmlinkage long sys_close(unsigned int fd);
asmlinkage long sys_close_range(unsigned int fd, unsigned int max_fd,
                                unsigned int flags);
asmlinkage long sys_vhangup(void);
asmlinkage long sys_get_sched_history(pid_t pid,
                                void __user *buf,
                                int max_events);

/* fs/pipe.c */
asmlinkage long sys_pipe2(int __user *fildes, int flags);
```

The file `syscalls.h` was updated by adding the declaration of the system call:

```
asmlinkage long sys_get_sched_history(pid_t pid, void __user *buf, int max_events);
```

This allows the kernel to recognize the system call interface.

4. Scheduler Modification (core.c)

```
/*
 * context_switch - switch to the new MM and the new thread's register state.
 */
extern void log_sched_event(pid_t pid, int event);

static __always_inline struct rq *
context_switch(struct rq *rq, struct task_struct *prev,
              struct task_struct *next, struct rq_flags *rf)
{
    prepare_task_switch(rq, prev, next);

    log_sched_event(prev->pid, 1); // OUT
    log_sched_event(next->pid, 0); // IN

    /*
     * For paravirt, this is coupled with an exit in switch_to to
     * combine the page table reload and the switch backend into
     * one hypercall.
     */
    arch_start_context_switch(prev);

    /*
     * kernel -> kernel    lazy + transfer active
     * user  -> kernel    lazy + mmgrab() active
     */
}
```

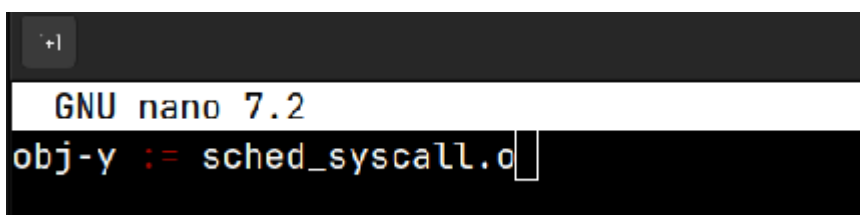
The kernel scheduler file `core.c` was modified to log scheduling events during context switches.

A call to the logging function `log_sched_event` was added inside the `context_switch` function:

- When a process is switched out → event recorded as OUT
- When a process is switched in → event recorded as IN

This ensures that every context switch is captured and stored in the buffer.

5. System Call Makefile



```
+|
GNU nano 7.2
obj-y := sched_syscall.o
```

A `Makefile` was created inside the `sched_syscall` directory:

obj-y := sched_syscall.o

This ensures that the system call source file is compiled into the kernel.

3. User Space Implementation

A user-space program `test.c` was developed to invoke the `get_sched_history` system call and retrieve scheduling events of a process.

The program defines the system call number and uses the `syscall()` interface to pass the process ID, a buffer, and the maximum number of events.

It generates scheduling activity and then retrieves the recorded events from the kernel.

The retrieved data is processed and displayed in a formatted manner, showing:

- Timestamp
- CPU core
- Process ID
- Event type (IN/OUT)

```

GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>

#define __NR_get_sched_history 451
#define MAX_EVENTS 1000

struct sched_event {
    unsigned long long timestamp;
    int cpu;
    int event; // 0 = IN, 1 = OUT
    int pid;
};

int main()
{
    struct sched_event buffer[MAX_EVENTS];

    int pid = getpid();
    printf("PID: %d\n\n", pid);

    // generate scheduling activity
    for (volatile long i = 0; i < 2000000000; i++);

    int ret = syscall(__NR_get_sched_history,
                     pid,
                     buffer,
                     MAX_EVENTS);

    if (ret <= 0) {
        printf("No events captured\n");
        return 0;
    }

    printf("%-12s %-5s %-6s %-5s\n", "Time(ns)", "CPU", "PID", "Event");

    int printed = 0;

    for (int i = 0; i < ret; i++) {
        if (buffer[i].pid == pid) {

            printf("%-12llu %-5d %-6d %-5s\n",
                  buffer[i].timestamp,
                  buffer[i].cpu,
                  buffer[i].pid,
                  buffer[i].event == 0 ? "IN" : "OUT");

            printed++;

            if (printed >= 6) break; // 3 IN/OUT pairs
        }
    }

    return 0;
}

```

```

        printed++;

        if (printed >= 6) break; // 3 IN/OUT pairs
    }

    return 0;
}

```

3.1 System Call Invocation

The system call is invoked using:

syscall(451, pid, buffer, MAX_EVENTS);

This returns the number of scheduling events captured for the given process.

3.2 Compiling and Executing the User program

```
(base) jai_wath_17@Curse:~/Documents/os_proj/user_space$ ./test
PID: 6912

No events captured
(base) jai_wath_17@Curse:~/Documents/os_proj/user_space$ ./test
PID: 6913

Time(ns)    CPU    PID    Event
167218202531 8      6913   OUT
167218224387 9      6913   IN
(base) jai_wath_17@Curse:~/Documents/os_proj/user_space$ ./test
PID: 6916

Time(ns)    CPU    PID    Event
169777198289 8      6916   OUT
169777203037 8      6916   IN
(base) jai_wath_17@Curse:~/Documents/os_proj/user_space$ killall yes
[1]-  Terminated                  yes > /dev/null
[2]+  Terminated                  yes > /dev/null
(base) jai_wath_17@Curse:~/Documents/os_proj/user_space$ yes > /dev/null &
yes > /dev/null &
yes > /dev/null &
[1] 6949
[2] 6950
[3] 6951
(base) jai_wath_17@Curse:~/Documents/os_proj/user_space$ ./test
PID: 6960

Time(ns)    CPU    PID    Event
246770201168 8      6960   OUT
246770208361 8      6960   IN
(base) jai_wath_17@Curse:~/Documents/os_proj/user_space$ ./test
PID: 6963

Time(ns)    CPU    PID    Event
249575553304 4      6963   OUT
249575558542 4      6963   IN
(base) jai_wath_17@Curse:~/Documents/os_proj/user_space$ ./test
PID: 6964

Time(ns)    CPU    PID    Event
252475933302 0      6964   OUT
252475948945 0      6964   IN
252475987984 0      6964   OUT
252476013195 0      6964   IN
252518597726 0      6964   OUT
252518612811 0      6964   IN
(base) jai_wath_17@Curse:~/Documents/os_proj/user_space$
```

When the program prints “**No events captured**”, it means that no scheduling events were recorded for that process. The system call only returns events that are logged during **context switches**, so if the process has not yet been switched in or out by the scheduler, the buffer remains empty.

The command `yes > /dev/null &` is used to generate scheduling activity. The `yes` command continuously produces output, creating a CPU-intensive process. The output is redirected to `/dev/null`, which discards it, preventing terminal clutter. The `&` runs the process in the background. This combination forces the CPU to stay busy.

Due to this increased CPU usage, the scheduler frequently performs **context switches** between processes. Each switch triggers the logging function added in the kernel, which records IN and OUT events. This is why, after running this command, scheduling events start appearing in the output.

In the final output, multiple IN and OUT events are shown for the same process along with timestamps and CPU IDs. This indicates that the process was scheduled multiple times on the CPU, demonstrating correct functioning of the system call and confirming that scheduling activity is being successfully captured.